

Pipeline de Datos del Macroentorno Ecuatoriano: Limpieza, Modelo Relacional y Dashboard Analítico

Martin Emanuel Ruiz Sanchez Carrera de Computación
 Universidad Técnica Particular de Loja
 Loja, Ecuador
 meruiz22@utpl.edu.ec

Resumen—Este artículo documenta el diseño e implementación de un pipeline de datos que integra nueve fuentes públicas del macroentorno ecuatoriano. El pipeline sigue el patrón medallón Bronze-Silver-Gold sobre Python y PostgreSQL. Extrae datos del Banco Central del Ecuador, el INEC, la Superintendencia de Compañías y el Ministerio de Educación. Los transforma con cuatro módulos de limpieza independientes y los carga en un modelo relacional de 14 tablas. Siete vistas Gold alimentan un dashboard en Power BI con tres páginas analíticas. El proceso se integra con un sistema de automatización RPA que deposita los datos en una tabla consolidada en formato JSON. El reto identifica problemas técnicos resueltos durante la implementación y propone extensiones para bases de datos heterogéneas y mayor escalabilidad ante nuevas fuentes.

Index Terms—pipeline de datos, arquitectura medallón, ETL, PostgreSQL, Power BI, macroentorno, CRISP-DM, RPA, limpieza de datos

I. INTRODUCCIÓN

El reto de Practicum 2.2 de 6to ciclo pide construir el Tablero Macroentorno de la Universidad Técnica Particular de Loja, que consolidará indicadores económicos y laborales de nueve fuentes públicas del Ecuador. Hoy ese proceso es manual: cuando cambia un archivo fuente, el conocimiento para actualizarlo vive en personas, no en código.

El reto resuelve ese problema en siete semanas. El pipeline parte de los archivos que entrega el equipo de RPA y termina en un dashboard con tres preguntas analíticas respondidas. El stack es Python, PostgreSQL y Power BI.

El dashboard responde tres preguntas concretas para la UTPL. Analiza la evolución económica ecuatoriana de los últimos 20 años con datos del BCE. identifica los sectores y provincias con mayor actividad económica y empleo usando VAB, ENEMDU y el Censo 2022. Cruza bachilleres por provincia (MINEDUC) con empresas activas (Supercias) para detectar zonas con demanda potencial de educación superior.

Construir ese pipeline requiere competencias que el mercado demanda. El Future of Jobs Report 2025 proyecta que la demanda de especialistas en Big Data crece más del 100 % entre 2025 y 2030 [1]. La UTPL forma a esos especialistas con datos reales, fuentes públicas y decisiones técnicas concretas.

El resto de este documento se organiza así. La Sección II define los conceptos que sostienen la arquitectura. La Sección III describe la arquitectura del pipeline. La Sección IV detalla la metodología aplicada. La Sección V revisa los trabajos relacionados. La Sección VI presenta el stack técnico. La

Sección VII discute las tendencias del campo para 2025–2030. La Sección VIII reporta los resultados obtenidos. La Sección IX identifica limitaciones y trabajo futuro. La Sección X presenta las conclusiones.

II. MARCO CONCEPTUAL

Esta sección define los conceptos que sostienen la arquitectura, desde el almacenamiento de datos hasta el pipeline que los mueve entre capas.

Data Warehouse. Repositorio central de datos históricos estructurados, integrados desde múltiples fuentes y optimizados para consultas analíticas [2]. Transforma los datos antes de cargarlos (ETL) y mantiene esquemas fijos.

Data Lake. Almacén de datos en formato crudo sobre almacenamiento de objetos de bajo costo [3]. Acepta datos estructurados, semi-estructurados y no estructurados sin transformación previa. Requiere metadatos activos para mantener la rastreabilidad.

Data Lakehouse. Arquitectura que une la flexibilidad del Data Lake con las garantías transaccionales del Data Warehouse [4]. Soporta cargas analíticas y de machine learning sin mover datos entre sistemas.

Arquitectura Medallón (Bronze-Silver-Gold). Patrón de tres zonas de calidad progresiva dentro de un Lakehouse [5]. Bronze guarda los datos en estado original. Silver aplica limpieza, tipado y normalización. Gold materializa vistas listas para el dashboard. En el reto, Bronze recibe los archivos de RPA, Silver los almacena en PostgreSQL con 14 tablas y FK correctas, y Gold expone siete vistas para Power BI.

ETL y ELT. ETL extrae, transforma y carga en ese orden [2]. ELT carga primero y transforma dentro del motor de destino [6]. El reto sigue ELT: cada módulo Python carga en PostgreSQL y las vistas Gold transforman ahí mismo.

Pipeline de datos. Secuencia automatizada de pasos que mueve y transforma datos desde las fuentes hasta el destino [7]. El pipeline del reto es reproducible: detecta registros nuevos automáticamente y los procesa sin intervención manual.

Modelo Entidad-Relación (ER). Representación gráfica de entidades, atributos y relaciones de un dominio [8]. Define la estructura antes de escribir código.

CRISP-DM. Metodología de referencia para proyectos de minería de datos y ciencia de datos. Define seis fases: comprensión del negocio, comprensión de los datos, preparación de los datos, modelado, evaluación y despliegue. El reto aplica esta

metodología de forma implícita: las semanas 1 a 4 corresponden a las tres primeras fases y las semanas 5 a 7 cubren modelado, evaluación y despliegue.

III. ARQUITECTURA DEL PIPELINE

La Fig. 1 muestra la arquitectura completa del pipeline. El diseño sigue el patrón medallón con tres capas y se integra con el proceso de automatización RPA.

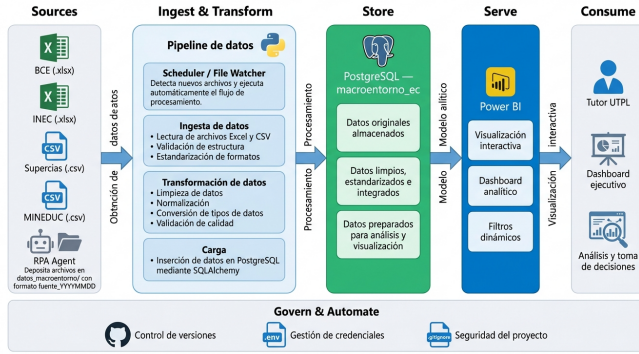


Figura 1. Arquitectura del pipeline del Macroentorno Ecuador. Bronze almacena los archivos de RPA. Python transforma y carga en PostgreSQL. Las vistas Gold alimentan Power BI.

La capa Bronze recibe los archivos sin modificarlos. El equipo de RPA los deposita en una tabla intermedia con trece indicadores distintos, en formato JSON. La capa Silver aplica limpieza y carga los datos en 14 tablas relacionales. La capa Gold expone siete vistas optimizadas para el dashboard.

El sistema RPA usa n8n para orquestar flujos automáticos, cada uno con una frecuencia de extracción ajustada al ritmo de actualización de su fuente. Cada flujo identifica el indicador, guarda su valor y calcula un código que detecta cambios, lo que evita reprocesar registros sin modificaciones.

IV. METODOLOGÍA

El reto aplica CRISP-DM en siete semanas. La primera semana cubre la comprensión del negocio: el diseño del diagrama ER y la definición de las tres preguntas del dashboard. Las semanas dos y tres cubren la comprensión y preparación de los datos, con la descarga de las nueve fuentes, la limpieza en cuatro módulos Python y la resolución de nueve problemas técnicos. La semana cuatro completa la preparación de datos con la carga en PostgreSQL, la creación de las 14 tablas y las siete vistas Gold, y el acuerdo de integración con RPA. Las semanas cinco y seis corresponden al modelado y la evaluación, con la construcción del dashboard en Power BI y la validación de sus tres páginas analíticas. La séptima semana cierra el despliegue: documentación técnica, configuración de la automatización y la demo final.

Las fuentes del reto incluyen cinco archivos del BCE (PIB real, PIB nominal, VAB cantonal, petróleo y IEE), dos del INEC (ENEMDU y Censo 2022), dos de Supercias (ranking y directorio) y uno de MINEDUC (AMIE 2023-2024). Cada fuente tiene un módulo Python independiente que encapsula su lógica de limpieza. Esta separación sigue el principio de responsabilidad única y facilita el mantenimiento cuando una fuente cambia su formato.

V. TRABAJOS RELACIONADOS

La Tabla I sintetiza los trabajos que fundamentan las decisiones técnicas del reto.

Cuadro I
TRABAJOS RELACIONADOS

Ref.	Aporte	Conexión con el reto
[3]	Lakehouse: storage y ACID	Bronze recibe archivos de RPA. Silver en PostgreSQL aplica ACID.
[4]	Sin metadatos el repositorio se degrada	Cada módulo documenta sus decisiones de limpieza.
[2]	Metodología dimensional de Kimball	Define las dimensiones compartidas y las 14 tablas Silver con FK.
[6]	ELT: cada fuente propietaria de su lógica	Cuatro módulos Python encapsulan la limpieza de su fuente.
[9]	RPA optimiza ingestión	El acuerdo de semana 4 fija subcarpeta, formato y canal de notificación.
[8]	Grafos ER como estándar de BD	El diagrama ER se aprueba en semana 1 antes de escribir código.
[10]	Párrafo interpretativo supera a la estética	6to ciclo exige un párrafo de análisis por página del dashboard.
[11]	Preparación de datos consume >70 % del tiempo	Las semanas 1 a 4 se dedican a limpieza y modelo.

Azzabi et al. [3] documentan la evolución del Lakehouse hacia una arquitectura con garantías transaccionales ACID. Jain et al. [12] compararon Delta Lake, Hudi e Iceberg: Delta lidera en velocidad de carga; Iceberg ofrece mayor interoperabilidad. Harby y Zulkernine [4] confirman que documentar el linaje de cada archivo separa una arquitectura madura de una improvisada. Nambiar y Mundra [2] revisan la metodología dimensional de Kimball en el contexto actual. Barret et al. [8] confirman que el diagrama ER es la herramienta estándar para comunicar un diseño relacional antes de implementarlo. Machado et al. [6] describen el paso de ETL a ELT en el contexto Data Mesh. Bhardwaj et al. [9] demuestran que RPA escala cuando los contratos de interfaz son precisos. Lai et al. [11] miden que la preparación de datos consume más del 70 % del tiempo en proyectos de BI. Tadakala et al. [10] demuestran que los dashboards con párrafo interpretativo superan en efectividad a los puramente visuales.

VI. HERRAMIENTAS Y STACK TÉCNICO

El reto define un stack de tres capas: Python transforma, PostgreSQL almacena y Power BI visualiza. Antes de que ese stack procese un solo archivo, el reto exige diseñar el modelo relacional que sostiene las tablas Silver.

La Fig. 2 presenta ese modelo ER. Muestra las 14 tablas con sus claves primarias y foráneas. Dos dimensiones, tiempo y geografía, se comparten entre las nueve fuentes. Las tablas de hechos referencian esas dimensiones con FK.

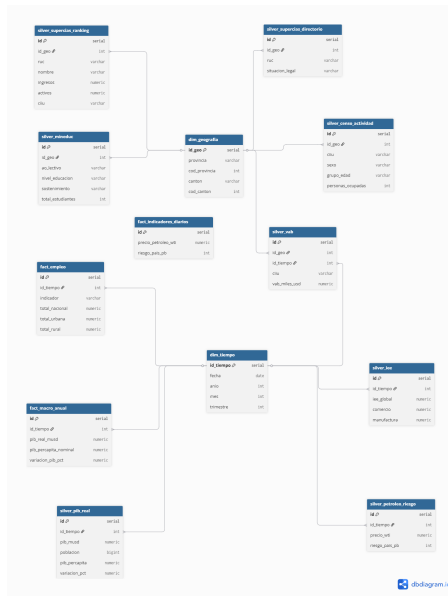


Figura 2. Modelo entidad-relación de la capa Silver del reto. Las dimensiones de tiempo y geografía se comparten entre las fuentes. Cada tabla de hechos referencia ambas con FK.

La Tabla II detalla el rol de cada herramienta.

Cuadro II
HERRAMIENTAS DEL STACK TÉCNICO

Herramienta	Uso
Python + pandas	Lee, limpia y transforma datos tabulares dentro de cuatro módulos de extracción y carga independientes.
PostgreSQL 18	Almacena las 14 tablas relacionales y expone siete vistas Gold mediante consultas SQL con soporte JSONB.
Power BI Desktop	Construye tres páginas analíticas y se conecta a PostgreSQL vía DirectQuery para mostrar datos actualizados.
GitHub	Controla versiones de los scripts Python, SQL y documentación del pipeline con repositorio privado.
n8n	Orquesta los flujos automáticos de RPA que depositan los datos en formato JSON.
draw.io	Genera los diagramas de arquitectura y el modelo ER antes de implementar el código.

Python procesa cada fuente con un módulo independiente. El módulo de INEC normaliza la estructura pivotada de ENEMDU. El módulo de MINEDUC lee el AMIE y calcula el total de bachilleres por género. El módulo del BCE convierte las fechas del IEE y carga los cinco archivos de esa fuente. El módulo de Supercias limpia el ranking y el directorio de empresas activas. Antes de la carga, un paso adicional adapta la sintaxis de Oracle a PostgreSQL en los datos que llegan del RPA. PostgreSQL recibe las 14 tablas Silver con FK correctas y expone siete vistas Gold. Power BI conecta esas vistas y construye las tres páginas del dashboard.

VII. TENDENCIAS Y FUTURO

La Tabla III resume cinco tendencias del campo de datos para 2025–2030.

Cuadro III
TENDENCIAS 2025–2030 EN EL CAMPO DE DATOS

Tendencia	Descripción
IA Generativa en pipelines	Automatiza partes de la limpieza y la imputación, pero el juicio sobre nulos y tipos de dato sigue siendo del desarrollador [11].
DataOps continuo	Convierte la calidad de dato en parte del flujo de entrega, con pruebas dentro del pipeline [13].
Convergencia RPA-Datos	RPA ingesta fuentes sin API. El contrato de interfaz entre equipos es el punto crítico [9].
Data Mesh	Trata cada fuente como un producto de dato con propietario y nivel de servicio definidos [6].
Dashboards con análisis ejecutivo	Un párrafo interpretativo por página comunica mejor las decisiones que un dashboard puramente visual [10].

La IA Generativa automatiza partes de la limpieza, pero la decisión de cómo normalizar una estructura pivotada o cómo tratar el primer año de variación nula del PIB real requiere juicio del desarrollador. Foidl et al. [13] identifican 41 factores que influyen en la calidad de un pipeline y ubican la limpieza como la principal fuente de errores. El WEF proyecta más del 100 % de crecimiento en demanda de especialistas en datos para 2030 [1].

VIII. RESULTADOS

El pipeline completó la carga de las nueve fuentes públicas sin errores. La Tabla IV presenta los conteos finales por tabla.

Cuadro IV
CONTEOS DE FILAS POR TABLA SILVER TRAS LA CARGA COMPLETA

Tabla	Filas cargadas
tab_consolidado	1.949.463
dim_tiempo	8.005
dim_geografia	247
pib_real	60 (1965–2024)
pib_nominal	60 (2000–2025)
petroleo_riesgo	7.962
iee	196
enemdu	1.872
vab_cantonal	16.575
censo_actividad	78.452
censo_ocupacion	39.226
mineduc_amie	16.250
supercias_directorio	223.797
Total	≈394.000 filas analíticas

La resolución de los nueve problemas técnicos consumió el mayor tiempo del proyecto. Tres de ellos surgieron por diferencias de sintaxis entre Oracle y PostgreSQL. Dos surgieron por inconsistencias en la estructura del JSON del RPA. Los cuatro restantes surgieron por características propias de cada fuente, como el sufijo provisional P en los años del BCE o la ausencia del campo `anio` en los registros VAB cantonal.

El dashboard generó siete vistas Gold, dos de ellas propuestas por el estudiante de 6to ciclo: una vista compara el IEE con el PIB, y otra cruza el VAB por sector económico. La primera

muestra que el IEE anticipa con un año de adelanto los cambios en la variación del PIB. La segunda identifica las provincias con mayor concentración económica y los sectores que las lideran.

La página P3 del dashboard produce la métrica de mayor valor estratégico para la UTPL. El ratio bachilleres por empresa activa identifica Zamora Chinchipe, Bolívar y Carchi como las provincias con mayor demanda potencial de educación superior.

La Fig. 3 muestra esa página del dashboard.

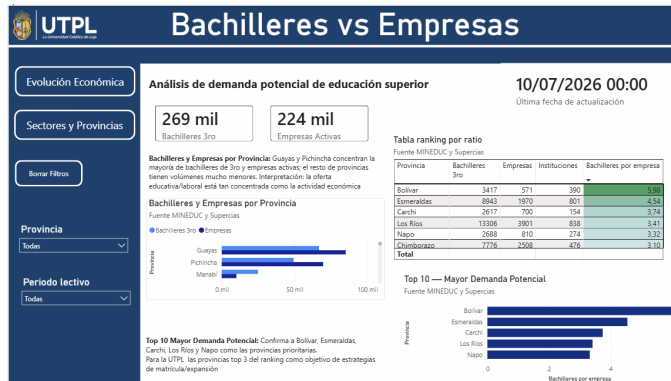


Figura 3. Página P3 del dashboard: bachilleres por provincia frente a empresas activas (Supercias). Zamora Chinchipe, Bolívar y Carchi muestran la mayor demanda potencial.

IX. LIMITACIONES Y SUGERENCIAS

El reto identifica dos limitaciones que abren trabajo futuro.

Fuente única de base de datos. El pipeline usa solo PostgreSQL. Fuentes como el SRI o el BCE distribuyen datos en Oracle, MongoDB o archivos Excel con macros. Un pipeline robusto necesita conectores para múltiples motores de base de datos. SQLAlchemy soporta Oracle, MySQL y SQLite con el mismo código. Agregar un conector Oracle permitiría consumir directamente las exportaciones del RPA sin convertir la sintaxis.

Escalabilidad para nuevas fuentes. Cada fuente nueva requiere un módulo de limpieza hecho a la medida. El pipeline no tiene un conector genérico que reduzca ese trabajo repetitivo. Un framework de ingesta configurable, con reglas de limpieza declarativas en lugar de código, aceleraría la incorporación de futuras fuentes.

Planes a futuro. Dos extensiones aumentan el valor del sistema. Primero, integrar la automatización de RPA directamente en el pipeline, en lugar de mantenerlos como dos sistemas separados que se comunican por archivos. Segundo, migrar a una plataforma unificada como Microsoft Fabric o Databricks, que reduciría la cantidad de herramientas distintas del stack.

X. CONCLUSIONES

X-A. Arquitectura para integrar fuentes heterogéneas

La arquitectura Lakehouse con patrón medallón Bronze-Silver-Gold organiza el pipeline en tres zonas de calidad progresiva [3], [5] (Fig. 1). Bronze recibe los archivos sin modificarlos. Python los transforma y los carga en PostgreSQL. Las vistas Gold alimentan Power BI. El repositorio GitHub con módulos versionados garantiza que cualquier miembro

del equipo reproduce el resultado. Harby y Zulkernine [4] confirman que documentar el linaje de cada decisión de limpieza es lo que diferencia un repositorio útil de uno inutilizable.

X-B. Modelo relacional diseñado antes de escribir código

La metodología dimensional de cuatro pasos revisada por Nambiar y Mundra [2] ordena el diseño del modelo relacional. El esquema resultante tiene dimensiones compartidas entre las fuentes y tablas de hechos con FK correctas (Fig. 2). Barret et al. [8] confirman que el diagrama ER comunica el diseño antes de que el código exista. Cada error detectado en el diagrama es un error que no aparece en la base de datos.

X-C. Stack técnico y limpieza de fuentes heterogéneas

Python procesa cada fuente con un módulo independiente que encapsula su lógica de limpieza. PostgreSQL almacena las tablas relacionales y expone vistas Gold listas para DirectQuery. Power BI construye tres páginas analíticas. Lai et al. [11] miden que más del 70 % del tiempo de un proyecto de BI va a preparación de datos. Dedicar las semanas 1 a 4 a la limpieza y al modelo antes de conectar la herramienta de visualización es la práctica que distingue proyectos de BI exitosos de los que fallan en producción.

X-D. Integración con automatización y perspectiva 2025-2030

La integración entre el pipeline y el sistema RPA requiere un contrato de interfaz preciso: indicador, formato JSON, dato clave y hash de contenido. Bhardwaj et al. [9] demuestran que la automatización escala cuando esos contratos son precisos. El pipeline detecta los registros nuevos automáticamente y los procesa sin intervención manual (Fig. 1). Para 2030, el Data Mesh y DataOps consolidan un modelo donde cada fuente es un producto de dato con propietario y SLA [6].

REFERENCIAS

- [1] World Economic Forum, *Future of Jobs Report 2025*. Ginebra: WEF, 2025. <https://www.weforum.org/publications/the-future-of-jobs-report-2025/>
- [2] A. Nambiar y D. Mundra, "An overview of data warehouse and data lake in modern enterprise data management," *Big Data Cogn. Comput.*, vol. 6, no. 4, p. 132, 2022. <https://doi.org/10.3390/bdcc6040132>
- [3] S. Azzabi, Z. Alfughi y A. Ouda, "Data lakes: A survey of concepts and architectures," *Computers*, vol. 13, no. 7, p. 183, 2024. <https://doi.org/10.3390/computers13070183>
- [4] A. A. Harby y F. Zulkernine, "From data warehouse to lakehouse: A comparative review," en *Proc. IEEE Int. Conf. Big Data*, Osaka, Japón, 2022, pp. 389–395. <https://doi.org/10.1109/BigData55660.2022.10020719>
- [5] Databricks, "What is medallion lakehouse architecture?" *Microsoft Azure Docs*, 2024. <https://learn.microsoft.com/en-us/azure/databricks/lakehouse/medallion>
- [6] I. A. Machado, C. Costa y M. Y. Santos, "Data mesh: Concepts and principles of a paradigm shift in data architectures," *Procedia Comput. Sci.*, vol. 196, pp. 263–271, 2022. <https://doi.org/10.1016/j.procs.2021.12.033>
- [7] J. Naso y C. Padden, *Data Platform Fundamentals*. Dagster Labs, 2024. <https://dagster.io/blog/data-platform-fundamentals>
- [8] N. Barret et al., "Entity/Relationship graphs: Principled design, modeling, and data integrity management," *Proc. ACM Manag. Data*, vol. 3, no. 1, 2025. <https://doi.org/10.1145/3709690>

- [9] V. Bhardwaj, A. Noonia, S. Chaurasia, M. Kumar, A. Rashid y M. T. Ben Othman, "Optimizing structured data processing through robotic process automation," *J. Eur. Syst. Autom.*, vol. 57, no. 5, pp. 1523–1530, 2024. <https://doi.org/10.18280/jesa.570528>
- [10] L. Tadakala et al., "Beyond visualization: Building decision intelligence through iterative dashboard refinement," arXiv:2510.27572, 2025. <https://arxiv.org/pdf/2510.27572>
- [11] E. Lai, Y. He y S. Chaudhuri, "Auto-Prep: Holistic prediction of data preparation steps for self-service BI," *Proc. VLDB Endow.*, vol. 14, no. 7, 2025. arXiv:2504.11627.
- [12] P. Jain, P. Kraft, C. Power, T. Das, I. Stoica y M. Zaharia, "Analyzing and comparing lakehouse storage systems," en *Proc. 13th CIDR*, 2023. <https://www.cidrdb.org/cidr2023/papers/p92-jain.pdf>
- [13] H. Foidl, V. Golendukhina, R. Ramler y M. Felderer, "Data pipeline quality: Influencing factors, root causes of data-related issues, and processing problem areas for developers," *J. Syst. Softw.*, vol. 207, p. 111855, 2024. <https://doi.org/10.1016/j.jss.2023.111855>